Assignment 2: CNNs

Liuyang Li | Part 3: Theory and calculations

For a convolution or pooling operation with dilation 1, the output along one spatial axis is

`N_out = floor((N + 2P - K) / S) + 1`.

Here N is the input size, P is padding on each side, K is the kernel size, and S is the stride. Height and width are calculated separately. A convolution's output depth equals its number of filters.

Question 1

Input: 32 x 32 x 3; 8 filters; kernel 5 x 5; stride 1; padding 0.

`H_out = W_out = floor((32 + 2(0) - 5) / 1) + 1 = 27 + 1 = 28`.

There are 8 filters, so the output depth is 8. **Output: 28 x 28 x 8.** The input depth of 3 is handled inside each filter; it does not multiply the number of output channels.

Question 2

With stride 1, same padding preserves height and width. A 5 x 5 kernel needs `P = (5 - 1) / 2 = 2` on each side.

`H_out = W_out = floor((32 + 2(2) - 5) / 1) + 1 = 31 + 1 = 32`.

The number of filters remains 8. **Output: 32 x 32 x 8.**

Question 3

Input: 64 x 64; kernel 3 x 3; stride 2; padding 0.

`H_out = W_out = floor((64 - 3) / 2) + 1 = floor(30.5) + 1 = 31`.

Output spatial size: 31 x 31. We round down because the kernel must fit completely inside the input.

Question 4

Input feature map: 16 x 16; pooling kernel 2 x 2; stride 2; padding 0.

`H_out = W_out = floor((16 - 2) / 2) + 1 = 7 + 1 = 8`.

Output: 8 x 8. Max-pooling preserves the number of channels; if the input has C channels, its output is 8 x 8 x C.

Question 5

Both layers use a 3 x 3 kernel, stride 1, and same padding, so each uses `P = 1`.

`Layer 1: floor((128 + 2(1) - 3) / 1) + 1 = 127 + 1 = 128`.

`Layer 2: floor((128 + 2(1) - 3) / 1) + 1 = 127 + 1 = 128`.

Final spatial shape: 128 x 128. The channel count is not specified in the question. If the second layer has F2 filters, the complete output shape is 128 x 128 x F2.

Question 6

`model.train()` sets the model and its submodules to training mode; it does not itself run training or update weights. A newly created PyTorch model is already in training mode. Therefore, removing this line will usually have no effect if the model has not previously been switched to evaluation mode.

If `model.eval()` was called earlier, removing `model.train()` leaves the model in evaluation mode. For architectures containing Dropout or BatchNorm, this changes their behavior: Dropout remains disabled, and BatchNorm uses stored running statistics without updating them. In the CNN specified here, Conv2d, ReLU, MaxPool2d, Flatten, and Linear behave the same in training and evaluation modes, so this mode change does not alter the forward calculation. Evaluation mode also does not disable gradients; that requires `no_grad()` or `inference_mode()`. Calling `model.train()` before each training phase is still good practice.

References

Assignment2.pdf, supplied assignment specifications and questions.

PyTorch, Modules: https://docs.pytorch.org/docs/stable/notes/modules.html

PyTorch, Autograd mechanics: https://docs.pytorch.org/docs/stable/notes/autograd.html